

Capítulo 2

Estilos CSS y modelos de caja

2.1 CSS y HTML

Como aclaramos anteriormente, la nueva especificación de HTML (HTML5) no describe solo los nuevos elementos HTML o el lenguaje mismo. La web demanda diseño y funcionalidad, no solo organización estructural o definición de secciones. En este nuevo paradigma, HTML se presenta junto con CSS y Javascript como un único instrumento integrado.

La función de cada tecnología ya ha sido explicada en capítulos previos, así como los nuevos elementos HTML responsables de la estructura del documento. Ahora es momento de analizar CSS, su relevancia dentro de esta unión estratégica y su influencia sobre la presentación de documentos HTML.

Oficialmente CSS nada tiene que ver con HTML5. CSS no es parte de la especificación y nunca lo fue. Este lenguaje es, de hecho, un complemento desarrollado para superar las limitaciones y reducir la complejidad de HTML. Al comienzo, atributos dentro de las etiquetas HTML proveían estilos esenciales para cada elemento, pero a medida que el lenguaje evolucionó, la escritura de códigos se volvió más compleja y HTML por sí mismo no pudo más satisfacer las demandas de diseñadores. En consecuencia, CSS pronto fue adoptado como la forma de separar la estructura de la presentación. Desde entonces, CSS ha crecido y ganado importancia, pero siempre desarrollado en paralelo, enfocado en las necesidades de los diseñadores y apartado del proceso de evolución de HTML.

La versión 3 de CSS sigue el mismo camino, pero esta vez con un mayor compromiso. La especificación de HTML5 fue desarrollada considerando CSS a cargo del diseño. Debido a esta consideración, la integración entre HTML y CSS es ahora vital para el desarrollo web y esta es la razón por la que cada vez que mencionamos HTML5 también estamos haciendo referencia a CSS3, aunque oficialmente se trate de dos tecnologías completamente separadas.

En este momento las nuevas características incorporadas en CSS3 están siendo implementadas e incluidas junto al resto de la especificación en navegadores compatibles con HTML5. En este capítulo, vamos a estudiar conceptos básicos de CSS y las nuevas técnicas de CSS3 ya disponibles para presentación y estructuración. También aprenderemos cómo utilizar los nuevos selectores y pseudo clases que hacen más fácil la selección e identificación de elementos HTML.

Conceptos básicos: CSS es un lenguaje que trabaja junto con HTML para proveer estilos visuales a los elementos del documento, como tamaño, color, fondo, bordes, etc...

IMPORTANTE: En este momento las nuevas incorporaciones de CSS3 están siendo implementadas en las últimas versiones de los navegadores más populares, pero algunas de ellas se encuentran aún en estado experimental. Por esta razón, estos nuevos estilos deberán ser precedidos por prefijos tales como `-moz-` o `-webkit-` para ser efectivamente interpretados. Analizaremos este importante asunto más adelante.

2.2 Estilos y estructura

A pesar de que cada navegador garantiza estilos por defecto para cada uno de los elementos HTML, estos estilos no necesariamente satisfacen los requerimientos de cada diseñador. Normalmente se encuentran muy distanciados de lo que queremos para nuestros sitios webs. Diseñadores y desarrolladores a menudo deben aplicar sus propios estilos para obtener la organización y el efecto visual que realmente desean.

IMPORTANTE: En esta parte del capítulo vamos a revisar estilos CSS y explicar algunas técnicas básicas para definir la estructura de un documento. Si usted ya se encuentra familiarizado con estos conceptos, siéntase libre de obviar las partes que ya conoce.

Elementos block

Con respecto a la estructura, básicamente cada navegador ordena los elementos por defecto de acuerdo a su tipo: *block* (bloque) o *inline* (en línea). Esta clasificación está asociada con la forma en que los elementos son mostrados en pantalla.

- **Elementos *block*** son posicionados uno sobre otro hacia abajo en la página.
- **Elementos *inline*** son posicionados lado a lado, uno al lado del otro en la misma línea, sin ningún salto de línea a menos que ya no haya más espacio horizontal para ubicarlos.

Casi todos los elementos estructurales en nuestros documentos serán tratados por los navegadores como elementos *block* por defecto. Esto significa que cada elemento HTML que representa una parte de la organización visual (por ejemplo, `<section>`, `<nav>`, `<header>`, `<footer>`, `<div>`) será posicionado debajo del anterior.

En el Capítulo 1 creamos un documento HTML con la intención de reproducir un sitio web tradicional. El diseño incluyó barras horizontales y dos columnas en el medio. Debido a la forma en que los navegadores muestran estos elementos por defecto, el resultado en la

pantalla está muy lejos de nuestras expectativas. Tan pronto como el archivo HTML con el código del Listado 1-18, Capítulo 1, es abierto en el navegador, la posición errónea en la pantalla de las dos columnas definidas por los elementos `<section>` y `<aside>` es claramente visible. Una columna está debajo de la otra en lugar de estar a su lado, como correspondería. Cada bloque (*block*) es mostrado por defecto tan ancho como sea posible, tan alto como la información que contiene y uno sobre otro, como se muestra en la Figura 2-1.

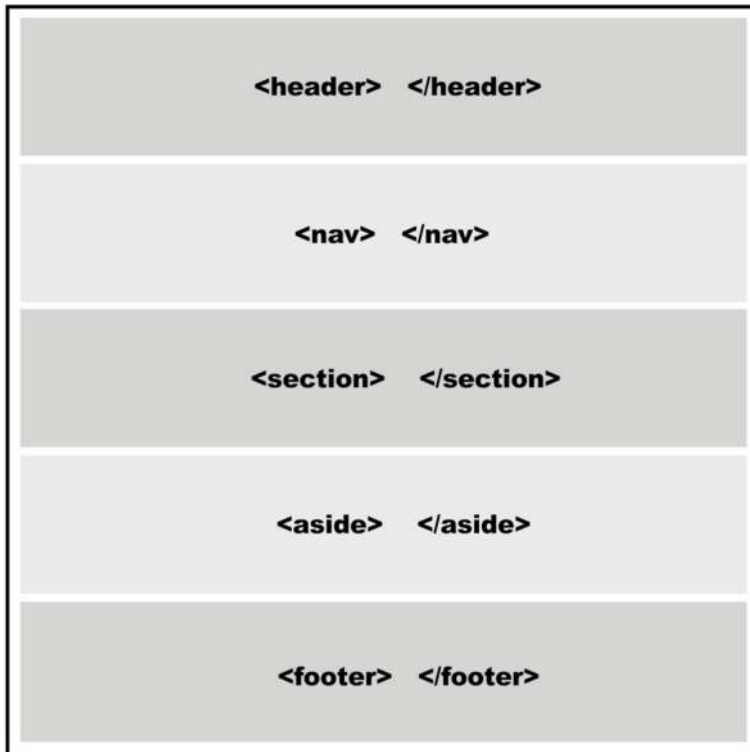


Figura 2-1. Representación visual de una página web mostrada con estilos por defecto.

Modelos de caja

Para aprender cómo podemos crear nuestra propia organización de los elementos en pantalla, debemos primero entender cómo los navegadores procesan el código HTML. Los navegadores consideran cada elemento HTML como una caja. Una página web es en realidad un grupo de cajas ordenadas siguiendo ciertas reglas. Estas reglas son establecidas por estilos provistos por los navegadores o por los diseñadores usando CSS.

CSS tiene un set predeterminado de propiedades destinados a sobrescribir los estilos provistos por navegadores y obtener la organización deseada. Estas propiedades no son específicas, tienen que ser combinadas para formar reglas que luego serán usadas para agrupar cajas y obtener la correcta disposición en pantalla. La combinación de estas reglas

es normalmente llamada modelo o sistema de disposición. Todas estas reglas aplicadas juntas constituyen lo que se llama un modelo de caja.

Existe solo un modelo de caja que es considerado estándar estos días, y muchos otros que aún se encuentran en estado experimental. El modelo válido y ampliamente adoptado es el llamado Modelo de Caja Tradicional, el cual ha sido usado desde la primera versión de CSS.

Aunque este modelo ha probado ser efectivo, algunos modelos experimentales intentan superar sus deficiencias, pero la falta de consenso sobre el reemplazo más adecuado aún mantiene a este viejo modelo en vigencia y la mayoría de los sitios webs programados en HTML5 lo continúan utilizando.

2.3 Conceptos básicos sobre estilos

Antes de comenzar a insertar reglas CSS en nuestro archivo de estilos y aplicar un modelo de caja, debemos revisar los conceptos básicos sobre estilos CSS que van a ser utilizados en el resto del libro.

Aplicar estilos a los elementos HTML cambia la forma en que estos son presentados en pantalla. Como explicamos anteriormente, los navegadores proveen estilos por defecto que en la mayoría de los casos no son suficientes para satisfacer las necesidades de los diseñadores. Para cambiar esto, podemos sobrescribir estos estilos con los nuestros usando diferentes técnicas.

Conceptos básicos: En este libro encontrará solo una introducción breve a los estilos CSS. Solo mencionamos las técnicas y propiedades que necesita conocer para entender los temas y códigos estudiados en próximos capítulos. Si considera que no tiene la suficiente experiencia en CSS y necesita mayor información visite nuestro sitio web y siga los enlaces correspondientes a este capítulo.

Hágalo usted mismo: Dentro de un archivo de texto vacío, copie cada código HTML estudiado en los siguientes listados y abra el archivo en su navegador para comprobar su funcionamiento. Tenga en cuenta que el archivo debe tener la extensión `.html` para ser abierto y procesado correctamente.

Estilos en línea

Una de las técnicas más simples para incorporar estilos CSS a un documento HTML es la de asignar los estilos dentro de las etiquetas por medio del atributo `style`.

El Listado 2-1 muestra un documento HTML simple que contiene el elemento `<p>` modificado por el atributo `style` con el valor `font-size: 20px`. Este estilo cambia el tamaño por defecto del texto dentro del elemento `<p>` a un nuevo tamaño de 20 pixeles.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este es el título del documento</title>
</head>
<body>
  <p style="font-size: 20px">Mi texto</p>
</body>
</html>
```

Listado 2-1. Estilos CSS dentro de etiquetas HTML.

Usar la técnica demostrada anteriormente es una buena manera de probar estilos y obtener una vista rápida de sus efectos, pero no es recomendado para aplicar estilos a todo el documento. La razón es simple: cuando usamos esta técnica, debemos escribir y repetir cada estilo en cada uno de los elementos que queremos modificar, incrementando el tamaño del documento a proporciones inaceptables y haciéndolo imposible de mantener y actualizar. Solo imagine lo que ocurriría si decide que en lugar de 20 pixeles el tamaño de cada uno de los elementos `<p>` debería ser de 24 pixeles. Tendría que modificar cada estilo en cada etiqueta `<p>` en el documento completo.

Estilos embebidos

Una mejor alternativa es insertar los estilos en la cabecera del documento y luego usar referencias para afectar los elementos HTML correspondientes:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <style>
    p { font-size: 20px }
  </style>
</head>
<body>
  <p>Mi texto</p>
</body>
</html>
```

Listado 2-2. Estilos listados en la cabecera del documento.

El elemento `<style>` (mostrado en el Listado 2-2) permite a los desarrolladores agrupar estilos CSS dentro del documento. En versiones previas de HTML era necesario

especificar qué tipo de estilos serían insertados. En HTML5 los estilos por defecto son CSS, por lo tanto no necesitamos agregar ningún atributo en la etiqueta de apertura `<style>`.

El código resaltado del Listado 2-2 tiene la misma función que la línea de código del Listado 2-1, pero en el Listado 2-2 no tuvimos que escribir el estilo dentro de cada etiqueta `<p>` porque todos los elementos `<p>` ya fueron afectados. Con este método, reducimos nuestro código y asignamos los estilos que queremos a elementos específicos utilizando referencias. Veremos más sobre referencias en este capítulo.

Archivos externos

Declarar los estilos en la cabecera del documento ahorra espacio y vuelve al código más consistente y actualizable, pero nos requiere hacer una copia de cada grupo de estilos en todos los documentos de nuestro sitio web. La solución es mover todos los estilos a un archivo externo y luego utilizar el elemento `<link>` para insertar este archivo dentro de cada documento que los necesite. Este método nos permite cambiar los estilos por completo simplemente incluyendo un archivo diferente. También nos permite modificar o adaptar nuestros documentos a cada circunstancia o dispositivo, como veremos al final del libro.

En el Capítulo 1, estudiamos la etiqueta `<link>` y cómo utilizarla para insertar archivos con estilos CSS en nuestros documentos. Utilizando la línea `<link rel="stylesheet" href="misestilos.css">` le decimos al navegador que cargue el archivo `misestilos.css` porque contiene todos los estilos necesarios para presentar el documento en pantalla. Esta práctica fue ampliamente adoptada por diseñadores que ya están trabajando con HTML5. La etiqueta `<link>` referenciando el archivo CSS será insertada en cada uno de los documentos que requieren de esos estilos:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
  <p>Mi texto</p>
</body>
</html>
```

Listado 2-3. *Aplicando estilos CSS desde un archivo externo.*

Hágalo usted mismo: De ahora en adelante agregaremos estilos CSS al archivo llamado `misestilos.css`. Debe crear este archivo en el mismo directorio (carpeta) donde se encuentra el archivo HTML y copiar los estilos CSS en su interior para comprobar cómo trabajan.

Conceptos básicos: Los archivos CSS son archivos de texto comunes. Al igual que los archivos HTML, puede crearlos utilizando cualquier editor de texto como el Bloc de Notas de Windows, por ejemplo.

Referencias

Almacenar todos nuestros estilos en un archivo externo e insertar este archivo dentro de cada documento que lo necesite es muy conveniente, sin embargo no podremos hacerlo sin buenos mecanismos que nos ayuden a establecer una específica relación entre estos estilos y los elementos del documento que van a ser afectados.

Cuando hablábamos sobre cómo incluir estilos en el documento, mostramos una de las técnicas utilizadas a menudo en CSS para referenciar elementos HTML. En el Listado 2-2, el estilo para cambiar el tamaño de la letra referenciaba cada elemento `<p>` usando la palabra clave `p`. De esta manera el estilo insertado entre las etiquetas `<style>` referenciaba cada etiqueta `<p>` del documento y asignaba ese estilo particular a cada una de ellas.

Existen varios métodos para seleccionar cuáles elementos HTML serán afectados por las reglas CSS:

- referencia por la palabra clave del elemento
- referencia por el atributo `id`
- referencia por el atributo `class`

Más tarde veremos que CSS3 es bastante flexible a este respecto e incorpora nuevas y más específicas técnicas para referenciar elementos, pero por ahora aplicaremos solo estas tres.

Referenciando con palabra clave

Al declarar las reglas CSS utilizando la palabra clave del elemento afectamos cada elemento de la misma clase en el documento. Por ejemplo, la siguiente regla cambiará los estilos de todos los elementos `<p>`:

```
p { font-size: 20px }
```

Listado 2-4. Referenciando por palabra clave.

Esta es la técnica presentada previamente en el Listado 2-2. Utilizando la palabra clave `p` al frente de la regla le estamos diciendo al navegador que esta regla debe ser aplicada a cada elemento `<p>` encontrado en el documento HTML. Todos los textos envueltos en etiquetas `<p>` tendrán el tamaño de 20 pixeles.

Por supuesto, lo mismo funcionará para cualquier otro elemento HTML. Si especificamos la palabra clave `span` en lugar de `p`, por ejemplo, cada texto entre etiquetas `<span>` tendrá un tamaño de 20 pixeles:

```
span { font-size: 20px }
```

Listado 2-5. Referenciando por otra palabra clave.

¿Pero qué ocurre si solo necesitamos referenciar una etiqueta específica? ¿Debemos usar nuevamente el atributo `style` dentro de esta etiqueta? La respuesta es no. Como aprendimos anteriormente, el método de **Estilos en Línea** (usando el atributo `style` dentro de etiquetas HTML) es una técnica en desuso y debería ser evitada. Para seleccionar un elemento HTML específico desde las reglas de nuestro archivo CSS, podemos usar dos atributos diferentes: `id` y `class`.

Referenciando con el atributo id

El atributo `id` es como un nombre que identifica al elemento. Esto significa que el valor de este atributo no puede ser duplicado. Este nombre debe ser único en todo el documento.

Para referenciar un elemento en particular usando el atributo `id` desde nuestro archivo CSS la regla debe ser declarada con el símbolo `#` al frente del valor que usamos para identificar el elemento:

```
#texto1 { font-size: 20px }
```

Listado 2-6. Referenciando a través del valor del atributo id.

La regla en el Listado 2-6 será aplicada al elemento HTML identificado con el atributo `id="texto1"`. Ahora nuestro código HTML lucirá de esta manera:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
  <p id="texto1">Mi texto</p>
</body>
</html>
```

Listado 2-7. Identificando el elemento <p> a través de su atributo id.

El resultado de este procedimiento es que cada vez que hacemos una referencia usando el identificador `texto1` en nuestro archivo CSS, el elemento con ese valor de `id`

será modificado, pero el resto de los elementos `<p>`, o cualquier otro elemento en el mismo documento, no serán afectados.

Esta es una forma extremadamente específica de referenciar un elemento y es normalmente utilizada para elementos más generales, como etiquetas estructurales. El atributo `id` y su especificidad es de hecho más apropiado para referencias en Javascript, como veremos en próximos capítulos.

Referenciando con el atributo `class`

La mayoría del tiempo, en lugar de utilizar el atributo `id` para propósitos de estilos es mejor utilizar `class`. Este atributo es más flexible y puede ser asignado a cada elemento HTML en el documento que comparte un diseño similar:

```
.texto1 { font-size: 20px }
```

Listado 2-8. Referenciando por el valor del atributo `class`.

Para trabajar con el atributo `class`, debemos declarar la regla CSS con un punto antes del nombre. La ventaja de este método es que insertar el atributo `class` con el valor `texto1` será suficiente para asignar estos estilos a cualquier elemento que queramos:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
  <p class="texto1">Mi texto</p>
  <p class="texto1">Mi texto</p>
  <p>Mi texto</p>
</body>
</html>
```

Listado 2-9. Asignando estilos a varios elementos a través del atributo `class`.

Los elementos `<p>` en las primeras dos líneas dentro del cuerpo del código en el Listado 2-9 tienen el atributo `class` con el valor `texto1`. Como dijimos previamente, la misma regla puede ser aplicada a diferentes elementos en el mismo documento. Por lo tanto, estos dos primeros elementos comparten la misma regla y ambos serán afectados por el estilo del Listado 2-8. El último elemento `<p>` conserva los estilos por defecto otorgados por el navegador.

El gran libro de HTML5, CSS3 y Javascript

La razón por la que debemos utilizar un punto delante del nombre de la regla es que es posible construir referencias más complejas. Por ejemplo, se puede utilizar el mismo valor para el atributo `class` en diferentes elementos pero asignar diferentes estilos para cada tipo:

```
p.texto1 { font-size: 20px }
```

Listado 2-10. Referenciando solo elementos `<p>` a través del valor del atributo `class`.

En el Listado 2-10 creamos una regla que referencia la clase llamada `texto1` pero solo para los elementos de tipo `<p>`. Si cualquier otro elemento tiene el mismo valor en su atributo `class` no será modificado por esta regla en particular.

Referenciando con cualquier atributo

Aunque los métodos de referencia estudiados anteriormente cubren un variado espectro de situaciones, a veces no son suficientes para encontrar el elemento exacto. La última versión de CSS ha incorporado nuevas formas de referenciar elementos HTML. Uno de ellas es el **Selector de Atributo**. Ahora podemos referenciar un elemento no solo por los atributos `id` y `class` sino también a través de cualquier otro atributo:

```
p[name] { font-size: 20px }
```

Listado 2-11. Referenciando solo elementos `<p>` que tienen el atributo `name`.

La regla en el Listado 2-11 cambia solo elementos `<p>` que tienen un atributo llamado `name`. Para imitar lo que hicimos previamente con los atributos `id` y `class`, podemos también especificar el valor del atributo:

```
p[name="mitexto"] { font-size: 20px }
```

Listado 2-12. Referenciando elementos `<p>` que tienen un atributo `name` con el valor `mitexto`.

CSS3 permite combinar `"=`" con otros para hacer una selección más específica:

```
p[name^="mi"] { font-size: 20px }  
p[name$="mi"] { font-size: 20px }  
p[name*="mi"] { font-size: 20px }
```

Listado 2-13. Nuevos selectores en CSS3.

Si usted conoce **Expresiones Regulares** desde otros lenguajes como Javascript o PHP, podrá reconocer los selectores utilizados en el Listado 2-13. En CSS3 estos selectores producen similares resultados:

- La regla con el selector ^= será asignada a todo elemento <p> que contiene un atributo **name** con un valor comenzado en “mi” (por ejemplo, “mitexto”, “micasa”).
- La regla con el selector \$= será asignada a todo elemento <p> que contiene un atributo **name** con un valor finalizado en “mi” (por ejemplo “textomi”, “casami”).
- La regla con el selector *= será asignada a todo elemento <p> que contiene un atributo **name** con un valor que incluye el texto “mi” (en este caso, el texto podría también encontrarse en el medio, como en “textomicasa”).

En estos ejemplos usamos el elemento <p>, el atributo **name**, y una cadena de texto al azar como “mi”, pero la misma técnica puede ser utilizada con cualquier atributo y valor que necesitemos. Solo tiene que escribir los corchetes e insertar entre ellos el nombre del atributo y el valor que necesita para referenciar el elemento HTML correcto.

Referenciando con pseudo clases

CSS3 también incorpora nuevas pseudo clases que hacen la selección aún más específica.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
  <div id="wrapper">
    <p class="mitexto1">Mi texto1</p>
    <p class="mitexto2">Mi texto2</p>
    <p class="mitexto3">Mi texto3</p>
    <p class="mitexto4">Mi texto4</p>
  </div>
</body>
</html>
```

Listado 2-14. Plantilla para probar pseudo clases.

Miremos por un momento el nuevo código HTML del Listado 2-14. Contiene cuatro elementos <p> que, considerando la estructura HTML, son hermanos entre sí e hijos del mismo elemento <div>.

Usando pseudo clases podemos aprovechar esta organización y referenciar un elemento específico sin importar cuánto conocemos sobre sus atributos y el valor de los mismos:

```
p:nth-child(2){
  background: #999999;
}
```

Listado 2-15. Pseudo clase *nth-child()*.

La pseudo clase es agregada usando dos puntos luego de la referencia y antes del su nombre. En la regla del Listado 2-15 referenciamos solo elementos `<p>`. Esta regla puede incluir otras referencias. Por ejemplo, podríamos escribirla como `.miclase:nth-child(2)` para referenciar todo elemento que es hijo de otro elemento y tiene el valor de su atributo `class` igual a `miclase`. La pseudo clase puede ser aplicada a cualquier tipo de referencia estudiada previamente.

La pseudo clase `nth-child()` nos permite encontrar un hijo específico. Como ya explicamos, el documento HTML del Listado 2-14 tiene cuatro elementos `<p>` que son hermanos. Esto significa que todos ellos tienen el mismo padre que es el elemento `<div>`. Lo que esta pseudo clase está realmente indicando es algo como: “el hijo en la posición...” por lo que el número entre paréntesis será el número de la posición del hijo, o índice. La regla del Listado 2-15 está referenciando cada segundo elemento `<p>` encontrado en el documento.

Hágalo usted mismo: Reemplace el código en su archivo HTML por el del Listado 2-14 y abra el archivo en su navegador. Incorpore las reglas estudiadas en el Listado 2-15 dentro del archivo `misestilos.css` para comprobar su funcionamiento.

Usando este método de referencia podemos, por supuesto, seleccionar cualquier hijo que necesitemos cambiando el número de índice. Por ejemplo, la siguiente regla tendrá impacto sólo sobre el último elemento `<p>` de nuestra plantilla:

```
p:nth-child(4){
  background: #999999;
}
```

Listado 2-16. Pseudo clase *nth-child()*.

Como seguramente se habrá dado cuenta, es posible asignar estilos a todos los elementos creando una regla para cada uno de ellos:

```
*{
  margin: 0px;
}
```

```
p:nth-child(1){
    background: #999999;
}
p:nth-child(2){
    background: #CCCCCC;
}
p:nth-child(3){
    background: #999999;
}
p:nth-child(4){
    background: #CCCCCC;
}
```

Listado 2-17. Creando una lista con la pseudo clase `nth-child()`.

La primera regla del Listado 2-17 usa el selector universal `*` para asignar el mismo estilo a cada elemento del documento. Este nuevo selector representa cada uno de los elementos en el cuerpo del documento y es útil cuando necesitamos establecer ciertas reglas básicas. En este caso, configuramos el margen de todos los elementos en 0 píxeles para evitar espacios en blanco o líneas vacías como las creadas por el elemento `<p>` por defecto.

En el resto del código del Listado 2-17 usamos la pseudo clase `nth-child()` para generar un menú o lista de opciones que son diferenciadas claramente en la pantalla por el color de fondo.

Hágalo usted mismo: Copie el último código dentro del archivo CSS y abra el documento HTML en su navegador para comprobar el efecto.

Para agregar más opciones al menú, podemos incorporar nuevos elementos `<p>` en el código HTML y nuevas reglas con la pseudo clase `nth-child()` usando el número de índice adecuado. Sin embargo, esta aproximación genera mucho código y resulta imposible de aplicar en sitios webs con contenido dinámico. Una alternativa para obtener el mismo resultado es aprovechar las palabras clave `odd` y `even` disponibles para esta pseudo clase:

```
*{
    margin: 0px;
}
p:nth-child(odd){
    background: #999999;
}
p:nth-child(even){
    background: #CCCCCC;
}
```

Listado 2-18. Aprovechando las palabras clave `odd` y `even`.

El gran libro de HTML5, CSS3 y Javascript

Ahora solo necesitamos dos reglas para crear la lista completa. Incluso si más adelante agregamos otras opciones, los estilos serán asignados automáticamente a cada una de ellas de acuerdo a su posición. La palabra clave `odd` para la pseudo clase `nth-child()` afecta los elementos `<p>` que son hijos de otro elemento y tienen un índice impar. La palabra clave `even`, por otro lado, afecta a aquellos que tienen un índice par.

Existen otras importantes pseudo clases relacionadas con esta última, como `first-child`, `last-child` y `only-child`, algunas de ellas recientemente incorporadas. La pseudo clase `first-child` referencia solo el primer hijo, `last-child` referencia solo el último hijo, y `only-child` afecta un elemento siempre y cuando sea el único hijo disponible. Estas pseudo clases en particular no requieren palabras clave o parámetros, y son implementadas como en el siguiente ejemplo:

```
*{
  margin: 0px;
}
p:last-child{
  background: #999999;
}
```

Listado 2-19. Usando `last-child` para modificar solo el último elemento `<p>` de la lista.

Otra importante pseudo clase llamada `not()` es utilizada realizar una negación:

```
:not(p){
  margin: 0px;
}
```

Listado 2-20. Aplicando estilos a cada elemento, excepto `<p>`.

La regla del Listado 2-20 asignará un margen de 0 píxeles a cada elemento del documento excepto los elementos `<p>`. A diferencia del selector universal utilizado previamente, la pseudo clase `not()` nos permite declarar una excepción. Los estilos en la regla creada con esta pseudo clase serán asignados a todo elemento excepto aquellos incluidos en la referencia entre paréntesis. En lugar de la palabra clave de un elemento podemos usar cualquier otra referencia que deseemos. En el próximo listado, por ejemplo, todos los elementos serán afectados excepto aquellos con el valor `mitexto2` en el atributo `class`:

```
:not(.mitexto2){
  margin: 0px;
}
```

Listado 2-21. Excepción utilizando el atributo `class`.

Cuando aplicamos la última regla al código HTML del Listado 2-14 el navegador asigna los estilos por defecto al elemento `<p>` identificado con el atributo `class` y el valor `mitexto2` y provee un margen de 0 pixeles al resto.

Nuevos selectores

Hay algunos selectores más que fueron agregados o que ahora son considerados parte de CSS3 y pueden ser útiles para nuestros diseños. Estos selectores usan los símbolos `>`, `+` y `~` para especificar la relación entre elementos.

```
div > p.mitexto2{
    color: #990000;
}
```

Listado 2-22. Selector `>`.

El selector `>` está indicando que el elemento a ser afectado por la regla es el elemento de la derecha cuando tiene al de la izquierda como su padre. La regla en el Listado 2-22 modifica los elementos `<p>` que son hijos de un elemento `<div>`. En este caso, fuimos bien específicos y referenciamos solamente el elemento `<p>` con el valor `mitexto2` en su atributo `class`.

El próximo ejemplo construye un selector utilizando el símbolo `+`. Este selector referencia al elemento de la derecha cuando es inmediatamente precedido por el de la izquierda. Ambos elementos deben compartir el mismo padre:

```
p.mitexto2 + p{
    color: #990000;
}
```

Listado 2-23. Selector `+`.

La regla del Listado 2-23 afecta al elemento `<p>` que se encuentra ubicado luego de otro elemento `<p>` identificado con el valor `mitexto2` en su atributo `class`. Si abre en su navegador el archivo HTML con el código del Listado 2-14, el texto en el tercer elemento `<p>` aparecerá en la pantalla en color rojo debido a que este elemento `<p>` en particular está posicionado inmediatamente después del elemento `<p>` identificado con el valor `mitexto2` en su atributo `class`.

El último selector que estudiaremos es el construido con el símbolo `~`. Este selector es similar al anterior pero el elemento afectado no necesita estar precediendo de inmediato al elemento de la izquierda. Además, más de un elemento puede ser afectado:

```
p.mitexto2 ~ p{
    color: #990000;
}
```

Listado 2-24. Selector ~.

La regla del Listado 2-24 afecta al tercer y cuarto elemento `<p>` de nuestra plantilla de ejemplo. El estilo será aplicado a todos los elementos `<p>` que son hermanos y se encuentran luego del elemento `<p>` identificado con el valor `mitexto2` en su atributo `class`. No importa si otros elementos se encuentran intercalados, los elementos `<p>` en la tercera y cuarta posición aún serán afectados. Puede verificar esto último insertando un elemento `<span>mitexto</span>` luego del elemento `<p>` que tiene el valor `mitexto2` en su atributo `class`. A pesar de este cambio solo los elementos `<p>` serán modificados por esta regla.

2.4 Aplicando CSS a nuestra plantilla

Como aprendimos más temprano en este mismo capítulo, todo elemento estructural es considerado una caja y la estructura completa es presentada como un grupo de cajas. Las cajas agrupadas constituyen lo que es llamado un Modelo de Caja.

Siguiendo con los conceptos básicos de CSS, vamos a estudiar lo que es llamado el Modelo de Caja Tradicional. Este modelo has sido implementado desde la primera versión de CSS y es actualmente soportado por cada navegador en el mercado, lo que lo ha convertido en un estándar para el diseño web.

Todo modelo, incluso aquellos aún en fase experimental, pueden ser aplicados a la misma estructura HTML, pero esta estructura debe ser preparada para ser afectada por estos estilos de forma adecuada. Nuestros documentos HTML deberán ser adaptados al modelo de caja seleccionado.

IMPORTANTE: El Modelo de Caja Tradicional presentado posteriormente no es una incorporación de HTML5, pero es introducido en este libro por ser el único disponible en estos momentos y posiblemente el que continuará siendo utilizado en sitios webs desarrollados en HTML5 durante los próximos años. Si usted ya conoce cómo implementarlo, siéntase en libertad de obviar esta parte del capítulo.

2.5 Modelo de caja tradicional

Todo comenzó con tablas. Las tablas fueron los elementos que sin intención se volvieron la herramienta ideal utilizada por desarrolladores para crear y organizar cajas de contenido en

la pantalla. Este puede ser considerado el primer modelo de caja de la web. Las cajas eran creadas expandiendo celdas y combinando filas de celdas, columnas de celdas y tablas enteras, unas sobre otras o incluso anidadas. Cuando los sitios webs crecieron y se volvieron más y más complejos esta práctica comenzó a presentar serios problemas relacionados con el tamaño y el mantenimiento del código necesario para crearlos.

Estos problemas iniciales hicieron necesario lo que ahora vemos como una práctica natural: la división entre estructura y presentación. Usando etiquetas `<div>` y estilos CSS fue posible reemplazar la función de tablas y efectivamente separar la estructura HTML de la presentación. Con elementos `<div>` y CSS podemos crear cajas en la pantalla, posicionar estas cajas a un lado o a otro y darles un tamaño, color o borde específico entre otras características. CSS provee propiedades específicas que nos permiten organizar las cajas acorde a nuestros deseos. Estas propiedades son lo suficientemente poderosas como para crear un modelo de caja que se transformó en lo que hoy conocemos como Modelo de Caja Tradicional.

Algunas deficiencias en este modelo mantuvieron a las tablas vivas por algún tiempo, pero los principales desarrolladores, influenciados por el suceso de las implementaciones Ajax y una cantidad enorme de nuevas aplicaciones interactivas, gradualmente volvieron a las etiquetas `<div>` y estilos CSS en un estándar. Finalmente el Modelo de Caja Tradicional fue adoptado a gran escala.

Plantilla

En el Capítulo 1 construimos una plantilla HTML5. Esta plantilla tiene todos los elementos necesarios para proveer estructura a nuestro documento, pero algunos detalles deben ser agregados para aplicar los estilos CSS y el Modelo de Caja Tradicional.

Este modelo necesita agrupar cajas juntas para ordenarlas horizontalmente. Debido a que el contenido completo del cuerpo es creado a partir de cajas, debemos agregar un elemento `<div>` para agruparlas, centrarlas y darles un tamaño específico.

La nueva plantilla lucirá de este modo:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="iso-8859-1">
  <meta name="description" content="Ejemplo de HTML5">
  <meta name="keywords" content="HTML5, CSS3, JavaScript">
  <title>Este texto es el título del documento</title>
  <link rel="stylesheet" href="misestilos.css">
</head>
<body>
<div id="agrupar">
  <header id="cabecera">
    <h1>Este es el título principal del sitio web</h1>
  </header>
```

```
<nav id="menu">
  <ul>
    <li>principal</li>
    <li>fotos</li>
    <li>videos</li>
    <li>contacto</li>
  </ul>
</nav>
<section id="seccion">
  <article>
    <header>
      <hgroup>
        <h1>Título del mensaje uno</h1>
        <h2>Subtítulo del mensaje uno</h2>
      </hgroup>
      <time datetime="2011-12-10" pubdate>publicado 10-12-2011
    </time>
    </header>
    Este es el texto de mi primer mensaje
    <figure>
      
      <figcaption>
        Esta es la imagen del primer mensaje
      </figcaption>
    </figure>
    <footer>
      <p>comentarios (0)</p>
    </footer>
  </article>
  <article>
    <header>
      <hgroup>
        <h1>Título del mensaje dos</h1>
        <h2>Subtítulo del mensaje dos</h2>
      </hgroup>
      <time datetime="2011-12-15" pubdate>publicado 15-12-2011
    </time>
    </header>
    Este es el texto de mi segundo mensaje
    <footer>
      <p>comentarios (0)</p>
    </footer>
  </article>
</section>
<aside id="columna">
  <blockquote>Mensaje número uno</blockquote>
  <blockquote>Mensaje número dos</blockquote>
</aside>
<footer id="pie">
  Derechos Reservados &copy; 2010-2011
</footer>
```

```
</div>
</body>
</html>
```

Listado 2-25. Nueva plantilla HTML5 lista para estilos CSS.

El Listado 2-25 provee una nueva plantilla lista para recibir los estilos CSS. Dos cambios importantes pueden distinguirse al comparar este código con el del Listado 1-18 del Capítulo 1. El primero es que ahora varias etiquetas fueron identificadas con los atributos `id` y `class`. Esto significa que podemos referenciar un elemento específico desde las reglas CSS con el valor de su atributo `id` o podemos modificar varios elementos al mismo tiempo usando el valor de su atributo `class`.

El segundo cambio realizado a la vieja plantilla es la adición del elemento `<div>` mencionado anteriormente. Este `<div>` fue identificado con el atributo y el valor `id="agrupar"`, y es cerrado al final del cuerpo con la etiqueta de cierre `</div>`. Este elemento se encarga de agrupar todos los demás elementos permitiéndonos aplicar el modelo de caja al cuerpo y designar su posición horizontal, como veremos más adelante.

Hágalo usted mismo: Compare el código del Listado 1-18 del Capítulo 1 con el código en el Listado 2-25 de este capítulo y ubique las etiquetas de apertura y cierre del elemento `<div>` utilizado para agrupar al resto. También compruebe cuáles elementos se encuentran ahora identificados con el atributo `id` y cuáles con el atributo `class`. Confirme que los valores de los atributos `id` son únicos para cada etiqueta. También necesitará reemplazar el código en el archivo HTML creado anteriormente por el del Listado 2-25 para aplicar los siguientes estilos CSS.

Con el documento HTML finalizado es tiempo de trabajar en nuestro archivo de estilos.

Selector universal *

Comencemos con algunas reglas básicas que nos ayudarán a proveer consistencia al diseño:

```
* {
  margin: 0px;
  padding: 0px;
}
```

Listado 2-26. Regla CSS general.

Normalmente, para la mayoría de los elementos, necesitamos personalizar los márgenes o simplemente mantenerlos al mínimo. Algunos elementos por defecto tienen márgenes que son diferentes de cero y en la mayoría de los casos demasiado amplios. A medida que avanzamos en la creación de nuestro diseño encontraremos que la mayoría

El gran libro de HTML5, CSS3 y Javascript

de los elementos utilizados deben tener un margen de 0 píxeles. Para evitar el tener que repetir estilos constantemente, podemos utilizar el selector universal.

La primera regla en nuestro archivo CSS, presentada en el Listado 2-26, nos asegura que todo elemento tendrá un margen interno y externo de 0 píxeles. De ahora en más solo necesitaremos modificar los márgenes de los elementos que queremos que sean mayores que cero.

Conceptos básicos: Recuerde que en HTML cada elemento es considerado como una caja. El margen (**margin**) es en realidad el espacio alrededor del elemento, el que se encuentra por fuera del borde de esa caja (el estilo **padding**, por otro lado, es el espacio alrededor del contenido del elemento pero dentro de sus bordes, como el espacio entre el título y el borde de la caja virtual formada por el elemento `<h1>` que contiene ese título). El tamaño del margen puede ser definido por lados específicos del elemento o todos sus lados a la vez. El estilo **margin: 0px** en nuestro ejemplo establece un margen 0 o nulo para cada elemento de la caja. Si el tamaño hubiese sido especificado en 5 píxeles, por ejemplo, la caja tendría un espacio de 5 píxeles de ancho en todo su contorno. Esto significa que la caja estaría separada de sus vecinas por 5 píxeles. Volveremos sobre este tema más adelante en este capítulo.

Hágalo usted mismo: Debemos escribir todas las reglas necesarias para otorgar estilo a nuestra plantilla en un archivo CSS. El archivo ya fue incluido dentro del código HTML por medio de la etiqueta `<link>`, por lo que lo único que tenemos que hacer es crear un archivo de texto vacío con nuestro editor de textos preferido, grabarlo con el nombre **misestilos.css** y luego copiar en su interior la regla del Listado 2-26 y todas las presentadas a continuación.

Nueva jerarquía para cabeceras

En nuestra plantilla usamos elementos `<h1>` y `<h2>` para declarar títulos y subtítulos de diferentes secciones del documento. Los estilos por defecto de estos elementos se encuentran siempre muy lejos de lo que queremos y además en HTML5 podemos reconstruir la jerarquía H varias veces en cada sección (como aprendimos en el capítulo anterior). El elemento `<h1>`, por ejemplo, será usado varias veces en el documento, no solo para el título principal de la página web como pasaba anteriormente sino también para secciones internas, por lo que tenemos que otorgarle los estilos apropiados:

```
h1 {  
    font: bold 20px verdana, sans-serif;  
}  
h2 {  
    font: bold 14px verdana, sans-serif;  
}
```

Listado 2-27. Agregando estilos para los elementos `<h1>` y `<h2>`.

La propiedad `font`, asignada a los elementos `<h1>` y `<h2>` en el Listado 2-27, nos permite declarar todos los estilos para el texto en una sola línea. Las propiedades que pueden ser declaradas usando `font` son: `font-style`, `font-variant`, `font-weight`, `font-size/line-height`, y `font-family` en este orden. Con estas reglas estamos cambiando el grosor, tamaño y tipo de letra del texto dentro de los elementos `<h1>` y `<h2>` a los valores que deseamos.

Declarando nuevos elementos HTML5

Otra regla básica que debemos declarar desde el comienzo es la definición por defecto de elementos estructurales de HTML5. Algunos navegadores aún no reconocen estos elementos o los tratan como elementos *inline* (en línea). Necesitamos declarar los nuevos elementos HTML5 como elementos *block* para asegurarnos de que serán tratados como regularmente se hace con elementos `<div>` y de este modo construir nuestro modelo de caja:

```
header, section, footer, aside, nav, article, figure, figcaption,
hgroup{
    display: block;
}
```

Listado 2-28. Regla por defecto para elementos estructurales de HTML5.

A partir de ahora, los elementos afectados por la regla del Listado 2-28 serán posicionados uno sobre otro a menos que especifiquemos algo diferente más adelante.

Centrando el cuerpo

El primer elemento que es parte del modelo de caja es siempre `<body>`. Normalmente, por diferentes razones de diseño, el contenido de este elemento debe ser posicionado horizontalmente. Siempre deberemos especificar el tamaño de este contenido, o un tamaño máximo, para obtener un diseño consistente a través de diferentes configuraciones de pantalla.

```
body {
    text-align: center;
}
```

Listado 2-29. Centrando el cuerpo.

Por defecto, la etiqueta `<body>` (como cualquier otro elemento *block*) tiene un valor de ancho establecido en 100%. Esto significa que el cuerpo ocupará el ancho completo de la ventana del navegador. Por lo tanto, para centrar la página en la pantalla necesitamos centrar el contenido dentro del cuerpo. Con la regla agregada en el Listado 2-29, todo lo

que se encuentra dentro de `<body>` será centrado en la ventana, centrando de este modo toda la página web.

Creando la caja principal

Siguiendo con el diseño de nuestra plantilla, debemos especificar un tamaño o tamaño máximo para el contenido del cuerpo. Como seguramente recuerda, en el Listado 2-25 en este mismo capítulo agregamos un elemento `<div>` a la plantilla para agrupar todas las cajas dentro del cuerpo. Este `<div>` será considerado la caja principal para la construcción de nuestro modelo de caja (este es el propósito por el que lo agregamos). De este modo, modificando el tamaño de este elemento lo hacemos al mismo tiempo para todos los demás:

```
#agrupar {  
  width: 960px;  
  margin: 15px auto;  
  text-align: left;  
}
```

Listado 2-30. Definiendo las propiedades de la caja principal.

La regla en el Listado 2-30 está referenciando por primera vez un elemento a través del valor de su atributo `id`. El carácter `#` le está diciendo al navegador que el elemento afectado por este conjunto de estilos tiene el atributo `id` con el valor `agrupar`.

Esta regla provee tres estilos para la caja principal. El primer estilo establece un valor fijo de 960 pixeles. Esta caja tendrá siempre un ancho de 960 pixeles, lo que representa un valor común para un sitio web estos días (los valores se encuentran entre 960 y 980 pixeles de ancho, sin embargo estos parámetros cambian constantemente a través del tiempo, por supuesto).

El segundo estilo es parte de lo que llamamos el Modelo de Caja Tradicional. En la regla previa (Listado 2-29), especificamos que el contenido del cuerpo sería centrado horizontalmente con el estilo `text-align: center`. Pero esto solo afecta contenido *inline*, como textos o imágenes. Para elementos *block*, como un `<div>`, necesitamos establecer un valor específico para sus márgenes que los adapta automáticamente al tamaño de su elemento padre. La propiedad `margin` usada para este propósito puede tener cuatro valores: superior, derecho, inferior, izquierdo, en este orden. Esto significa que el primer valor declarado en el estilo representa el margen de la parte superior del elemento, el segundo es el margen de la derecha, y así sucesivamente. Sin embargo, si solo escribimos los primeros dos parámetros, el resto tomará los mismos valores. En nuestro ejemplo estamos usando esta técnica.

En el Listado 2-30, el estilo `margin: 15px auto` asigna 15 pixeles al margen superior e inferior del elemento `<div>` que está afectando y declara como automático el tamaño de los márgenes de izquierda y derecha (los dos valores declarados son usados para definir los cuatro márgenes). De esta manera, habremos generado un espacio de 15

pixeles en la parte superior e inferior del cuerpo y los espacios a los laterales (margen izquierdo y derecho) serán calculados automáticamente de acuerdo al tamaño del cuerpo del documento y el elemento `<div>`, efectivamente centrando el contenido en pantalla.

La página web ya está centrada y tiene un tamaño fijo de 960 pixeles. Lo próximo que necesitamos hacer es prevenir un problema que ocurre en algunos navegadores. La propiedad `text-align` es hereditaria. Esto significa que todos los elementos dentro del cuerpo y su contenido serán centrados, no solo la caja principal. El estilo asignado a `<body>` en el Listado 2-29 será asignado a cada uno de sus hijos. Debemos retornar este estilo a su valor por defecto para el resto del documento. El tercer y último estilo incorporado en la regla del Listado 2-30 (`text-align: left`) logra este propósito. El resultado final es que el contenido del cuerpo es centrado pero el contenido de la caja principal (el `<div>` identificado como **agrupar**) es alineado nuevamente hacia la izquierda, por lo tanto todo el resto del código HTML dentro de esta caja hereda este estilo.

Hágalo usted mismo: Si aún no lo ha hecho, copie cada una de las reglas listadas hasta este punto dentro de un archivo de texto vacío llamado `misestilos.css`. Este archivo debe estar ubicado en el mismo directorio (carpeta) que el archivo HTML con el código del Listado 2-25. Al terminar, deberá contar con dos archivos, uno con el código HTML y otro llamado `misestilos.css` con todos los estilos CSS estudiados desde el Listado 2-26. Abra el archivo HTML en su navegador y en la pantalla podrá notar la caja creada.

La cabecera

Continuemos con el resto de los elementos estructurales. Siguiendo la etiqueta de apertura del `<div>` principal se encuentra el primer elemento estructural de HTML5: `<header>`. Este elemento contiene el título principal de nuestra página web y estará ubicado en la parte superior de la pantalla. En nuestra plantilla, `<header>` fue identificado con el atributo `id` y el valor `cabecera`.

Como ya mencionamos, cada elemento *block*, así como el cuerpo, por defecto tiene un valor de ancho del 100%. Esto significa que el elemento ocupará todo el espacio horizontal disponible. En el caso del cuerpo, ese espacio es el ancho total de la pantalla visible (la ventana del navegador), pero en el resto de los elementos el espacio máximo disponible estará determinado por el ancho de su elemento padre. En nuestro ejemplo, el espacio máximo disponible para los elementos dentro de la caja principal será de 960 pixeles, porque su padre es la caja principal la cual fue previamente configurada con este tamaño.

```
#cabecera {  
  background: #FFFBB9;  
  border: 1px solid #999999;  
  padding: 20px;  
}
```

Listado 2-31. Agregando estilos para `<header>`.

Debido a que `<header>` ocupará todo el espacio horizontal disponible en la caja principal y será tratado como un elemento *block* (y por esto posicionada en la parte superior de la página), lo único que resta por hacer es asignar estilos que nos permitirán reconocer el elemento cuando es presentado en pantalla. En la regla mostrada en el Listado 2-31 le otorgamos a `<header>` un fondo amarillo, un borde sólido de 1 pixel y un margen interior de 20 pixeles usando la propiedad `padding`.

Barra de navegación

Siguiendo al elemento `<header>` se encuentra el elemento `<nav>`, el cual tiene el propósito de proporcionar ayuda para la navegación. Los enlaces agrupados dentro de este elemento representarán el menú de nuestro sitio web. Este menú será una simple barra ubicada debajo de la cabecera. Por este motivo, del mismo modo que el elemento `<header>`, la mayoría de los estilos que necesitamos para posicionar el elemento `<nav>` ya fueron asignados: `<nav>` es un elemento *block* por lo que será ubicado debajo del elemento previo, su ancho por defecto será 100% por lo que será tan ancho como su padre (el `<div>` principal), y (también por defecto) será tan alto como su contenido y los márgenes predeterminados. Por lo tanto, lo único que nos queda por hacer es mejorar su aspecto en pantalla. Esto último lo logramos agregando un fondo gris y un pequeño margen interno para separar las opciones del menú del borde del elemento:

```
#menu {
  background: #CCCCCC;
  padding: 5px 15px;
}
#menu li {
  display: inline-block;
  list-style: none;
  padding: 5px;
  font: bold 14px verdana, sans-serif;
}
```

Listado 2-32. Agregando estilos para `<nav>`.

En el Listado 2-32, la primera regla referencia al elemento `<nav>` por su atributo `id`, cambia su color de fondo y agrega márgenes internos de `5px` y `15px` con la propiedad `padding`.

Conceptos básicos: La propiedad `padding` trabaja exactamente como `margin`. Cuatro valores pueden ser especificados: superior, derecho, inferior, izquierdo, en este orden. Si solo declaramos un valor, el mismo será asignado para todos los espacios alrededor del contenido del elemento. Si en cambio especificamos dos valores, entonces el primero será asignado como margen interno de la parte superior e inferior del contenido y el segundo valor será asignado al margen interno de los lados, izquierdo y derecho.

Dentro de la barra de navegación hay una lista creada con las etiquetas `<ul>` y `<li>`. Por defecto, los ítems de una lista son posicionados unos sobre otros. Para cambiar este comportamiento y colocar cada opción del menú una al lado de la otra, referenciamos los elementos `<li>` dentro de este elemento `<nav>` en particular usando el selector `#menu li`, y luego asignamos a todos ellos el estilo `display: inline-block` para convertirlos en lo que se llama cajas *inline*. A diferencia de los elementos *block*, los elementos afectados por el parámetro `inline-block` estandarizado en CSS3 no generan ningún salto de línea pero nos permiten tratarlos como elementos *block* y así declarar un valor de ancho determinado. Este parámetro también ajusta el tamaño del elemento de acuerdo con su contenido cuando el valor del ancho no fue especificado.

En esta última regla también eliminamos el pequeño gráfico generado por defecto por los navegadores delante de cada opción del listado utilizando la propiedad `list-style`.

Section y aside

Los siguientes elementos estructurales en nuestro código son dos cajas ordenadas horizontalmente. El Modelo de Caja Tradicional es construido sobre estilos CSS que nos permiten especificar la posición de cada caja. Usando la propiedad `float` podemos posicionar estas cajas del lado izquierdo o derecho de acuerdo a nuestras necesidades. Los elementos que utilizamos en nuestra plantilla HTML para crear estas cajas son `<section>` y `<aside>`, cada uno identificado con el atributo `id` y los valores `seccion` y `columna` respectivamente.

```
#seccion {
  float: left;
  width: 660px;
  margin: 20px;
}
#columna {
  float: left;
  width: 220px;
  margin: 20px 0px;
  padding: 20px;
  background: #CCCCCC;
}
```

Listado 2-33. Creando dos columnas con la propiedad `float`.

La propiedad de CSS `float` es una de las propiedades más ampliamente utilizadas para aplicar el Modelo de Caja Tradicional. Hace que el elemento flote hacia un lado o al otro en el espacio disponible. Los elementos afectados por `float` actúan como elementos *block* (con la diferencia de que son ubicados de acuerdo al valor de esta propiedad y no el flujo normal del documento). Los elementos son movidos a izquierda o derecha en el área disponible, tanto como sea posible, respondiendo al valor de `float`.

Con las reglas del Listado 2-33 declaramos la posición de ambas cajas y sus respectivos tamaños, generando así las columnas visibles en la pantalla. La propiedad `float` mueve la caja al espacio disponible del lado especificado por su valor, `width` asigna un tamaño horizontal y `margin`, por supuesto, declara el margen del elemento.

Afectado por estos valores, el contenido del elemento `<section>` estará situado a la izquierda de la pantalla con un tamaño de 660 píxeles, más 40 píxeles de margen, ocupando un espacio total de 700 píxeles de ancho.

La propiedad `float` del elemento `<aside>` también tiene el valor `left` (izquierda). Esto significa que la caja generada será movida al espacio disponible a su izquierda. Debido a que la caja previa creada por el elemento `<section>` fue también movida a la izquierda de la pantalla, ahora el espacio disponible será solo el que esta caja dejó libre. La nueva caja quedará ubicada en la misma línea que la primera pero a su derecha, ocupando el espacio restante en la línea, creando la segunda columna de nuestro diseño.

El tamaño declarado para esta segunda caja fue de 220 píxeles. También agregamos un fondo gris y configuramos un margen interno de 20 píxeles. Como resultado final, el ancho de esta caja será de 220 píxeles más 40 píxeles agregados por la propiedad `padding` (los márgenes de los lados fueron declarados a `0px`).

Conceptos básicos: El tamaño de un elemento y sus márgenes son agregados para obtener el valor real ocupado en pantalla. Si tenemos un elemento de 200 píxeles de ancho y un margen de 10 píxeles a cada lado, el área real ocupada por el elemento será de 220 píxeles. El total de 20 píxeles del margen es agregado a los 200 píxeles del elemento y el valor final es representado en la pantalla. Lo mismo pasa con las propiedades `padding` y `border`. Cada vez que agregamos un borde a un elemento o creamos un espacio entre el contenido y el borde usando `padding` esos valores serán agregados al ancho del elemento para obtener el valor real cuando el elemento es mostrado en pantalla. Este valor real es calculado con la fórmula: **tamaño + márgenes + márgenes internos + bordes**.

Hágalo usted mismo: Lea el código del Listado 2-25. Controle cada regla CSS creada hasta el momento y busque en la plantilla los elementos HTML correspondientes a cada una de ellas. Siga las referencias, por ejemplo las claves de los elementos (como `h1`) y los atributos `id` (como `cabecera`), para entender cómo trabajan las referencias y cómo los estilos son asignados a cada elemento.

Footer

Para finalizar la aplicación del Modelo de Caja Tradicional, otra propiedad CSS tiene que ser aplicada al elemento `<footer>`. Esta propiedad devuelve al documento su flujo normal y nos permite posicionar `<footer>` debajo del último elemento en lugar de a su lado:

```
#pie {  
  clear: both;
```

```

text-align: center;
padding: 20px;
border-top: 2px solid #999999;
}

```

Listado 2-34. Otorgando estilos a `<footer>` y recuperando el normal flujo del documento.

La regla del Listado 2-34 declara un borde de 2 píxeles en la parte superior de `<footer>`, un margen interno (**padding**) de 20 píxeles, y centra el texto dentro del elemento. A sí mismo, restaura el normal flujo del documento con la propiedad **clear**. Esta propiedad simplemente restaura las condiciones normales del área ocupada por el elemento, no permitiéndole posicionarse adyacente a una caja flotante. El valor usualmente utilizado es **both**, el cual significa que ambos lados del elemento serán restaurados y el elemento seguirá el flujo normal (este elemento ya no es flotante como los anteriores). Esto, para un elemento *block*, quiere decir que será posicionado debajo del último elemento, en una nueva línea.

La propiedad **clear** también empuja los elementos verticalmente, haciendo que las cajas flotantes ocupen un área real en la pantalla. Sin esta propiedad, el navegador presenta el documento en pantalla como si los elementos flotantes no existieran y las cajas se superponen.

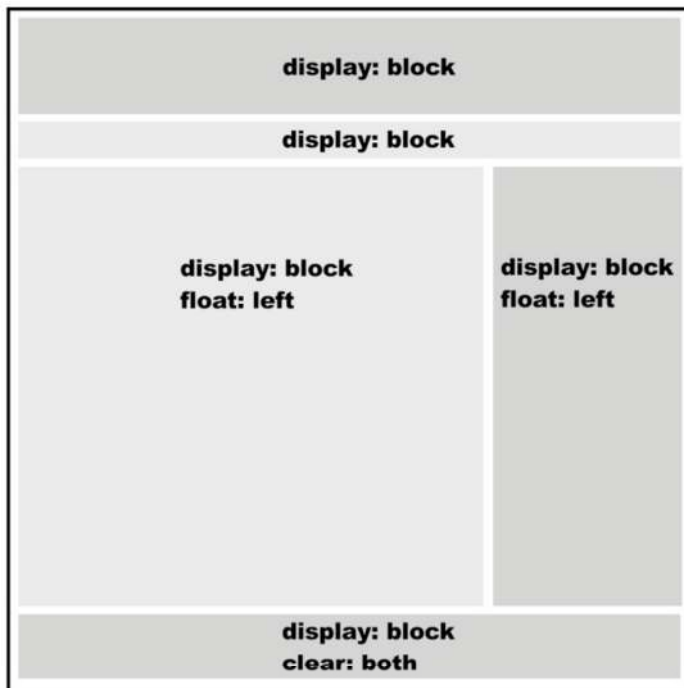


Figura 2-2. Representación visual del modelo de caja tradicional.

El gran libro de HTML5, CSS3 y Javascript

Cuando tenemos cajas posicionadas una al lado de la otra en el Modelo de Caja Tradicional siempre necesitamos crear un elemento con el estilo `clear: both` para poder seguir agregando otras cajas debajo de un modo natural. La Figura 2-2 muestra una representación visual de este modelo con los estilos básicos para lograr la correcta disposición en pantalla.

Los valores `left` (izquierda) y `right` (derecha) de la propiedad `float` no significan que las cajas deben estar necesariamente posicionadas del lado izquierdo o derecho de la ventana. Lo que los valores hacen es volver flotante ese lado del elemento, rompiendo el flujo normal del documento. Si el valor es `left`, por ejemplo, el navegador tratará de posicionar el elemento del lado izquierdo en el espacio disponible. Si hay espacio disponible luego de otro elemento, este nuevo elemento será situado a su derecha, porque su lado izquierdo fue configurado como flotante. El elemento flota hacia la izquierda hasta que encuentra algo que lo bloquea, como otro elemento o el borde de su elemento padre. Esto es importante cuando queremos crear varias columnas en la pantalla. En este caso cada columna tendrá el valor `left` en la propiedad `float` para asegurar que cada columna estará continua a la otra en el orden correcto. De este modo, cada columna flotará hacia la izquierda hasta que es bloqueada por otra columna o el borde del elemento padre.

Últimos toques

Lo único que nos queda por hacer es trabajar en el diseño del contenido. Para esto, solo necesitamos configurar los pocos elementos HTML5 restantes:

```
article {
  background: #FFFBCC;
  border: 1px solid #999999;
  padding: 20px;
  margin-bottom: 15px;
}
article footer {
  text-align: right;
}
time {
  color: #999999;
}
figcaption {
  font: italic 14px verdana, sans-serif;
}
```

Listado 2-35. *Agregando los últimos toques a nuestro diseño básico.*

La primera regla del Listado 2-35 referencia todos los elementos `<article>` y les otorga algunos estilos básicos (color de fondo, un borde sólido de 1 pixel, margen interno

y margen inferior). El margen inferior de 15 pixeles tiene el propósito de separar un elemento `<article>` del siguiente verticalmente.

Cada elemento `<article>` cuenta también con un elemento `<footer>` que muestra el número de comentarios recibidos. Para referenciar un elemento `<footer>` dentro de un elemento `<article>`, usamos el selector `article footer` que significa “cada `<footer>` dentro de un `<article>` será afectado por los siguientes estilos”. Esta técnica de referencia fue aplicada aquí para alinear a la derecha el texto dentro de los elementos `<footer>` de cada `<article>`.

Al final del código en el Listado 2-35 cambiamos el color de cada elemento `<time>` y diferenciamos la descripción de la imagen (insertada con el elemento `<figcaption>`) del resto del texto usando una tipo de letra diferente.

Hágalo usted mismo: Si aún no lo ha hecho, copie cada regla CSS listada en este capítulo desde el Listado 2-26, una debajo de otra, dentro del archivo `misestilos.css`, y luego abra el archivo HTML con la plantilla creada en el Listado 2-25 en su navegador. Esto le mostrará cómo funciona el Modelo de Caja Tradicional y cómo los elementos estructurales son organizados en pantalla.

IMPORTANTE: Puede acceder a estos códigos con un solo clic desde nuestro sitio web. Visite www.minkbooks.com.

Box-sizing

Existe una propiedad adicional incorporada en CSS3 relacionada con la estructura y el Modelo de Caja Tradicional. La propiedad `box-sizing` nos permite cambiar cómo el espacio total ocupado por un elemento en pantalla será calculado forzando a los navegadores a incluir en el ancho original los valores de las propiedades `padding` y `border`.

Como explicamos anteriormente, cada vez que el área total ocupada por un elemento es calculada, el navegador obtiene el valor final por medio de la siguiente fórmula: **tamaño + márgenes + márgenes internos + bordes**.

Por este motivo, si declaramos la propiedad `width` igual a 100 pixeles, `margin` en 20 pixeles, `padding` en 10 pixeles y `border` en 1 pixel, el área horizontal total ocupada por el elemento será: $100 + 40 + 20 + 2 = 162$ pixeles (note que tuvimos que duplicar los valores de `margin`, `padding` y `border` en la fórmula porque consideramos que los mismos fueron asignados tanto para el lado derecho como el izquierdo).

Esto significa que cada vez que declare el ancho de un elemento con la propiedad `width`, deberá recordar que el área real para ubicar el elemento en pantalla será seguramente más grande.

Dependiendo de sus costumbres, a veces podría resultar útil forzar al navegador a incluir los valores de `padding` y `border` en el tamaño del elemento. En este caso la nueva fórmula sería simplemente: **tamaño + márgenes**.

```
div {
  width: 100px;
  margin: 20px;
  padding: 10px;
  border: 1px solid #000000;

  -moz-box-sizing: border-box;
  -webkit-box-sizing: border-box;
  box-sizing: border-box;
}
```

Listado 2-36. Incluyendo padding y border en el tamaño del elemento.

La propiedad `box-sizing` puede tomar dos valores. Por defecto es configurada como `content-box`, lo que significa que los navegadores agregarán los valores de `padding` y `border` al tamaño especificado por `width` y `height`. Usando el valor `border-box` en su lugar, este comportamiento es cambiado de modo que `padding` y `border` son incluidos dentro del elemento.

El Listado 2-36 muestra la aplicación de esta propiedad en un elemento `<div>`. Este es solo un ejemplo y no vamos a usarlo en nuestra plantilla, pero puede ser útil para algunos diseñadores dependiendo de qué tan familiarizados se encuentran con los métodos tradicionales propuestos por versiones previas de CSS.

IMPORTANTE: En este momento, la propiedad `box-sizing`, al igual que otras importantes propiedades CSS3 estudiadas en próximos capítulos, se encuentra en estado experimental en algunos navegadores. Para aplicarla efectivamente a sus documentos, debe declararla con los correspondientes prefijos, como hicimos en el Listado 2-36. Los prefijos para los navegadores más comunes son los siguientes:

- `-moz-` para Firefox.
- `-webkit-` para Safari y Chrome.
- `-o-` para Opera.
- `-khtml-` para Konqueror.
- `-ms-` para Internet Explorer.
- `-chrome-` específico para Google Chrome.

2.6 Referencia rápida

En HTML5 la responsabilidad por la presentación de la estructura en pantalla está más que nunca en manos de CSS. Incorporaciones y mejoras se han hecho en la última versión de CSS para proveer mejores formas de organizar documentos y trabajar con sus elementos.

Selector de atributo y pseudo clases

CSS3 incorpora nuevos mecanismos para referenciar elementos HTML.

Selector de Atributo Ahora podemos utilizar otros atributos además de `id` y `class` para encontrar elementos en el documento y asignar estilos. Con la construcción `palabraclave[atributo=valor]`, podemos referenciar un elemento que tiene un atributo particular con un valor específico. Por ejemplo, `p[name="texto"]` referenciará cada elemento `<p>` con un atributo llamado `name` y el valor `"texto"`. CSS3 también provee técnicas para hacer esta referencia aún más específica. Usando las siguientes combinaciones de símbolos `^=`, `$=` y `*=` podemos encontrar elementos que comienzan con el valor provisto, elementos que terminan con ese valor y elementos que tienen el texto provisto en alguna parte del valor del atributo. Por ejemplo, `p[name^="texto"]` será usado para encontrar elementos `<p>` que tienen un atributo llamado `name` con un valor que comienza por `"texto"`.

Pseudo Clase :nth-child() Esta pseudo clase encuentra un hijo específico siguiendo la estructura de árbol de HTML. Por ejemplo, con el estilo `span:nth-child(2)` estamos referenciando el elemento `<span>` que tiene otros elementos `<span>` como hermanos y está localizado en la posición 2. Este número es considerado el índice. En lugar de un número podemos usar las palabras clave `odd` y `even` para referenciar elementos con un índice impar o par respectivamente (por ejemplo, `span:nth-child(odd)`).

Pseudo Clase :first-child Esta pseudo clase es usada para referenciar el primer hijo, similar a `:nth-child(1)`.

Pseudo Clase :last-child Esta pseudo clase es usada para referenciar el último hijo.

Pseudo Clase :only-child Esta pseudo clase es usada para referenciar un elemento que es el único hijo disponible de un mismo elemento padre.

Pseudo Clase :not() Esta pseudo clase es usada para referenciar todo elemento excepto el declarado entre paréntesis.

Selectores

CSS3 también incorpora nuevos selectores que ayudan a llegar a elementos difíciles de referenciar utilizando otras técnicas.

Selector > Este selector referencia al elemento de la derecha cuando tiene el elemento de la izquierda como padre. Por ejemplo, `div > p` referenciará cada elemento `<p>` que es hijo de un elemento `<div>`.

Selector + Este selector referencia elementos que son hermanos. La referencia apuntará al elemento de la derecha cuando es inmediatamente precedido por el de la izquierda. Por ejemplo, `span + p` afectará a los elementos `<p>` que son hermanos y están ubicados luego de un elemento `<span>`.

El gran libro de HTML5, CSS3 y Javascript

Selector ~ Este selector es similar al anterior, pero en este caso el elemento de la derecha no tiene que estar ubicado inmediatamente después del de la izquierda.